

Unidad 4: Agilidad en el Desarrollo de Software

Semana 1: Fundamentos de la Agilidad y el Framework Scrum

1) Introducción: ¿por qué ser ágiles hoy?

El desarrollo de software en las décadas de 1970, 1980 y 1990 se concibió bajo una mentalidad de la ingeniería tradicional (civil o industrial). La premisa era que, al igual que la construcción de un puente o un edificio, un proyecto de software podía ser completamente planificado, diseñado y presupuestado de antemano con una expectativa de resultados inmutables. Este enfoque se basaba en la idea de minimizar el riesgo congelando los requisitos.

Sin embargo, esta analogía falló fundamentalmente. A diferencia de la materia física, el software es **intangible**, lo que dificulta su visualización y la detección temprana de errores. Más crucialmente, el contexto empresarial y tecnológico es inherentemente **cambiante**. Los requisitos de negocio evolucionan, el mercado introduce nuevas competencias, y la tecnología (como la IA en el contexto de 2026) avanza a una velocidad que supera los ciclos de desarrollo tradicionales.

El modelo de "planificar todo y ejecutar después" generó proyectos que, aunque entregaban el software especificado, a menudo fallaban en entregar el **valor** esperado, ya que la necesidad original del negocio había caducado o mutado para el momento de la entrega. Por ello, la **agilidad** no debe confundirse con la simple velocidad. La agilidad es, por definición, la **capacidad de una organización y sus equipos para adaptarse al cambio** de manera efectiva y eficiente, reaccionando a la retroalimentación del cliente y a las condiciones del mercado con un bajo costo de corrección.

2) El pasado: el modelo en Cascada (Waterfall)

El modelo **Waterfall (Cascada)** fue durante años el paradigma dominante en ingeniería de software. Su lógica es **lineal y secuencial**: una fase debe completarse y validarse por completo antes de pasar a la siguiente. La intención era imponer disciplina y orden, pero esa rigidez introdujo un riesgo clave: el **"punto de no retorno"**.

2.1 Las fases secuenciales de Waterfall

1. **Requisitos:** La etapa más crítica, donde se recopilan, analizan y documentan exhaustivamente todas las necesidades del sistema. El resultado es un Documento de Especificación de Requisitos (DER) voluminoso, que el cliente debe firmar, congelando el alcance.
2. **Análisis:** Los requisitos se transforman en modelos y especificaciones técnicas para que el equipo de desarrollo comprenda la arquitectura y las tecnologías necesarias.

3. **Diseño:** Se define la arquitectura del software, la base de datos, las interfaces de usuario y los módulos internos. Se crean los planos detallados antes de la construcción.
4. **Codificación (Implementación):** Finalmente, los programadores escriben el código basándose estrictamente en los diseños y análisis previos.
5. **Pruebas (Verificación):** Se valida el software para asegurar que cumple con todos los requisitos definidos en la Fase 1. Esta etapa se realiza, a menudo, meses o años después de que se definieron los requisitos.
6. **Mantenimiento:** El producto se entrega al cliente y comienza la fase de soporte, corrección de errores no detectados y adaptación a nuevas plataformas.

2.2 El problema astronómico del retrabajo

El mayor defecto del modelo Waterfall radica en su estructura lineal. Si durante la fase de "Pruebas" se descubre que un requisito fue mal entendido, o si el cliente ha cambiado de opinión debido a una nueva tendencia del mercado, la corrección requiere volver a la Fase 1 o 2. El costo de este **retrabajo** es exponencialmente mayor en las etapas tardías. El tiempo invertido en la documentación inicial y la rigidez del plan significan que el proyecto a menudo llega tarde, fuera de presupuesto y, lo que es peor, entregando un producto que ya no satisface la necesidad real de negocio, sino una necesidad de meses atrás.

3) El nacimiento de la revolución: el Manifiesto Ágil

Ante la frustración generalizada con los procesos de desarrollo "pesados" (heavyweight), un grupo de 17 líderes de opinión en el desarrollo de software se reunió en la estación de esquí Snowbird, Utah, en febrero de 2001. Su objetivo no era crear una nueva metodología, sino una declaración de principios que sirviera como un faro para cualquier enfoque que priorizara la entrega rápida y la adaptación.

El resultado fue el **Manifiesto por el Desarrollo Ágil de Software**, un documento que es una **declaración de valores** y no un recetario. Establece un nuevo paradigma basado en cuatro pares de valores fundamentales que priorizan la interacción y el producto sobre el proceso y el papeleo.

3.1 Los 4 valores fundamentales (con profundización)

Valor Priorizado (Izquierda)	Sobre (Derecha)	Profundización
Individuos e interacciones	sobre procesos y herramientas	La comunicación efectiva y el talento humano son el motor principal del desarrollo. Un equipo cohesionado que resuelve problemas conversando supera la rigidez de un proceso burocrático o la dependencia de una herramienta de gestión compleja.

Software funcionando	sobre documentación extensiva	La medida de progreso no son los documentos de requisitos (que pueden estar desactualizados al mes), sino una versión del software que el usuario puede tocar, probar y criticar. La prioridad es entregar valor real en forma de funcionalidad.
Colaboración con el cliente	sobre negociación contractual	El cliente es el experto en el negocio y debe ser un participante activo y constante en el proceso de desarrollo. El foco pasa de defender un contrato rígido a trabajar juntos como socios para encontrar la mejor solución, aceptando que los requisitos pueden y deben cambiar.
Respuesta ante el cambio	sobre seguir un plan	Un plan (como el de Waterfall) es una foto estática de un momento. El mercado es dinámico. El enfoque ágil ve el cambio como una ventaja competitiva para el cliente. El plan se convierte en una guía flexible que se ajusta iterativamente en ciclos cortos.

3.2 La frase clave del Manifiesto (y lo que realmente significa)

La esencia del manifiesto se resume en: "Aunque valoramos los elementos de la derecha, **valoramos más los de la izquierda**". Esto no significa que la documentación, los procesos o los planes no existan. Significa que, ante un conflicto de prioridades, se debe elegir la opción que favorece la entrega de valor y la capacidad de adaptación

4) Los 12 principios ágiles (análisis operativo)

El Manifiesto se complementa con 12 principios que actúan como directrices operacionales para cualquier equipo que se considere ágil. Estos principios definen el comportamiento y la mentalidad necesarios para sostener los 4 valores.

1. **Satisfacción del cliente:** Se logra mediante la **entrega temprana y continua** de software que tenga valor para su negocio. Esto implica que el cliente no espera al final para ver el producto, sino que recibe partes funcionales de forma regular.
2. **Aceptación de cambios:** Fomentar una cultura donde los cambios, incluso en las fases tardías, son bienvenidos. Los procesos ágiles están diseñados para **aprovechar el cambio** y convertirlo en una ventaja sobre la competencia.
3. **Entrega frecuente:** Entregar software que funcione en ciclos cortos, que varían desde un par de

semanas hasta dos meses. Esto minimiza el riesgo, maximiza el *feedback* y permite una corrección rápida.

4. **Trabajo conjunto:** Los expertos de negocio (el cliente o *stakeholders*) y los desarrolladores deben colaborar **diariamente** a lo largo de todo el proyecto, eliminando la división tradicional entre "negocio" y "TI".
5. **Motivación:** El éxito reside en las personas. Se debe construir el proyecto en torno a **individuos motivados**, proporcionándoles el entorno, las herramientas y la confianza que necesitan para realizar su trabajo sin microgestión.
6. **Conversación cara a cara:** Históricamente, el método más efectivo para comunicar información dentro del equipo de desarrollo. Aunque hoy se ha adaptado a **videollamadas y herramientas síncronas** de alta calidad, el espíritu es priorizar la comunicación rica, directa y sin ambigüedades del diálogo en vivo.
7. **Software funcional:** Es la única **medida principal de progreso**. Los informes de estado, los documentos o las horas trabajadas son métricas secundarias; lo que importa es si el producto está entregando una funcionalidad operativa.
8. **Desarrollo sostenible:** Los procesos ágiles buscan que los patrocinadores, desarrolladores y usuarios puedan mantener un **ritmo constante de forma indefinida**. Esto es una lucha directa contra el agotamiento (*burnout*) y las largas jornadas laborales innecesarias.
9. **Excelencia técnica:** La atención continua a la **excelencia técnica** (código limpio, bien probado) y al **buen diseño** es lo que potencia la agilidad. Un código de baja calidad ralentiza futuros cambios.
10. **Simpleza:** El arte de **maximizar la cantidad de trabajo no realizado** es esencial. Esto significa no construir funcionalidad innecesaria, no crear documentación excesiva y centrarse solo en lo que entrega valor en el momento.
11. **Autogestión:** Las mejores arquitecturas, requisitos y diseños provienen de **equipos auto-organizados**. Los equipos deben tener la autonomía y la responsabilidad para decidir cómo lograr mejor el objetivo del proyecto.
12. **Mejora continua:** A **intervalos regulares** (ej. al final de un ciclo), el equipo debe reflexionar sobre su trabajo, sobre cómo ser más efectivos, y **ajustar su comportamiento** en el ciclo siguiente. Esto cierra el ciclo de la agilidad.

5) Agilidad Aumentada por IA: el paradigma "2.0"

La agilidad tradicional, surgida a principios de los años 2000, se centraba en optimizar la comunicación humana y reducir el desperdicio en procesos manuales. Sin embargo, en el contexto actual, la integración de la Inteligencia Artificial (IA) ha dado lugar a lo que se denomina **Agilidad Aumentada**.

En este nuevo paradigma, la agilidad no se limita a la gestión de reuniones o tableros, sino que se redefine a través de la **velocidad de iteración**. La capacidad de un equipo para pivotar ante cambios del mercado se ve multiplicada por la capacidad de la IA para procesar información, generar código base y automatizar tareas administrativas que antes consumían días

5.1 Relectura de los valores ágiles bajo el impacto de IA

- **Individuos e interacciones sobre procesos y herramientas:** La IA no reemplaza al individuo, sino que lo potencia (*Augmented Human*), permitiendo que la interacción humana se centre en la creatividad y la resolución de problemas complejos, delegando la ejecución repetitiva a la máquina.
- **Software funcionando sobre documentación extensiva:** Gracias a herramientas generativas, la documentación técnica puede ser creada de forma síncrona al desarrollo del código, eliminando la fricción entre "hacer" y "documentar".
- **Respuesta ante el cambio sobre seguir un plan:** Los modelos predictivos permiten simular escenarios y ajustar los Sprints con una base de datos más sólida, facilitando decisiones estratégicas rápidas.

6) Ingeniería de Requerimientos: el cimiento del desarrollo

Para evitar el caos, es clave distinguir claramente los tipos de necesidades que el software debe satisfacer. La IA puede aportar mucho en esta etapa: permite pasar de ideas abstractas a especificaciones más precisas, y facilita descomponer requerimientos complejos en tareas técnicas más pequeñas.

Con prompts estructurados, un analista puede pedir a la IA, por ejemplo:

- generación de matrices de trazabilidad,
- casos de uso,
- descomposición en tareas,
- borradores de criterios de aceptación y pruebas.

Esto reduce el esfuerzo manual y también el sesgo de interpretación.

6.1 Requisitos funcionales y no funcionales

Un requerimiento es una necesidad documentada sobre lo que un sistema debe ser o hacer.

- **Requisitos Funcionales (RF):** Definen las acciones específicas que el sistema debe ser capaz de realizar sin importar cómo se implementen. Representan el "qué" del software.
 - *Ejemplo:* "El sistema debe permitir al usuario filtrar productos por rango de precio".
- **Requisitos No Funcionales (RNF):** Definen los atributos de calidad, restricciones y propiedades del sistema. Se centran en el "cómo" se comporta el software en términos de rendimiento, seguridad, usabilidad y escalabilidad.
 - *Ejemplo:* "La plataforma debe soportar 10.000 usuarios concurrentes sin degradar el tiempo de respuesta por encima de los 2 segundos".

6.2 Historias de Usuario (User Stories)

En lugar de escribir manuales técnicos gigantes, los equipos ágiles usamos **Historias de Usuario**. Es una forma de describir una funcionalidad desde la perspectiva de quien la usa.

La Fórmula Estándar:

Como [rol del usuario], **quiero** [realizar una acción], **para** [obtener un beneficio/valor].

El Rol de la IA en la redacción de Historias:

Escribir 50 historias de usuario a mano es tedioso. La IA puede generar la estructura base para que el equipo solo tenga que revisarla y corregirla.

6.3 Criterios de Aceptación: definición de “Done”

Un error común del programador novato es decir "ya terminé" cuando el código solo funciona en su computadora. Los **Criterios de Aceptación** son la lista de chequeo que evita discusiones.

Ejemplo de Criterios de Aceptación para un Login:

1. El usuario ve un mensaje de error si la contraseña es incorrecta.
2. El sistema redirige al Home si los datos son válidos.
3. El campo de contraseña debe ocultar los caracteres por seguridad.

6.4 Comparación de apoyo con IA (según tu esquema)

Tarea Tradicional	Con IA (Agile 2.0)	Tiempo Ahorrado
Pensar requerimientos	Lluvia de ideas con IA	70%
Escribir Historias de Usuario	Generación y revisión	80%
Definir casos de prueba	Generación automática	90%

7) Scrum: el marco de trabajo dominante

Si la agilidad es la filosofía y la mentalidad, **Scrum** es el *framework* ligero más popular y utilizado para aplicar esa filosofía. Es un marco de trabajo para gestionar proyectos complejos, especialmente de desarrollo de productos, basado en la **teoría del control de procesos empírico**. Esto significa que Scrum no asume que se conoce todo de antemano (como Waterfall), sino que el **conocimiento proviene de la experiencia** y las decisiones se toman en función de lo que se observa.

7.1 Los pilares de Scrum (empirismo)

Estos tres pilares son la base del empirismo en Scrum y guían todas sus ceremonias y roles:

- **Transparencia:** Todos los aspectos significativos del proceso deben ser visibles para aquellos que son responsables del resultado. Esto incluye la definición de "terminado" (*Definition of Done*), el estado del trabajo y los impedimentos.
- **Inspección:** Los artefactos de Scrum y el progreso hacia un objetivo deben ser inspeccionados frecuentemente y con diligencia. Esto no es solo inspeccionar el producto, sino también el proceso de trabajo del equipo.
- **Adaptación:** Si el resultado de una inspección revela que uno o más aspectos se desvían de los límites aceptables, el proceso o el material que se está procesando debe ser ajustado de inmediato. La adaptación es el cambio en acción.

7.2 Roles en Scrum (responsabilidad y foco)

Scrum define tres roles específicos, cada uno con un conjunto claro de responsabilidades y sin jerarquías tradicionales. Forman el **Equipo Scrum**.

1. Product Owner (PO):

- **Enfoque Principal:** Maximizar el valor del producto resultante del trabajo del Equipo de Desarrollo.
- **Responsabilidades:** Es el único responsable de la gestión del *Product Backlog*. Esto incluye definir los ítems (historias de usuario, características), ordenarlos para lograr mejor los objetivos (priorización), asegurar que sean transparentes y claros para el equipo, y representar la voz del cliente y de los *stakeholders*. El PO decide *qué* se hace.
- **Habilidad Clave:** Conocimiento profundo del negocio y la capacidad de decir "no" a las peticiones que no maximizan el valor.

2. Scrum Master (SM):

- **Enfoque Principal:** Servir al Equipo Scrum y a la organización para aplicar y comprender la teoría, las prácticas y las reglas de Scrum.
- **Responsabilidades:** No es un jefe, sino un **líder servicial**. Se encarga de eliminar los impedimentos (obstáculos) que frenan al equipo, facilitar las ceremonias de Scrum para asegurar que sean productivas, entrenar al Product Owner y a la organización en prácticas ágiles, y proteger al equipo de interrupciones externas.
- **Habilidad Clave:** Liderazgo, *coaching* y resolución de conflictos.

3. Developers (Equipo de Desarrollo):

- **Enfoque Principal:** Crear el **Incremento** (la pieza de software funcional) en cada Sprint.
- **Responsabilidades:** Seleccionar las tareas del *Sprint Backlog*, planificar el trabajo diario para lograr el *Sprint Goal*, responsabilizarse de la calidad técnica y seguir la *Definition of Done*.
- **Característica Clave:** Son **Multidisciplinarios** (poseen todas las habilidades necesarias para convertir un ítem del *backlog* en un incremento funcional: programación, diseño, prueba, etc.) y **Autogestionados** (deciden internamente quién hace qué y cómo).

8) Eventos de Scrum (ceremonias): propósito y resultado

El trabajo en Scrum se organiza en ciclos de tiempo fijos y cortos llamados **Sprints** (de 1 a 4 semanas de duración). Estos ciclos son el latido del corazón de Scrum y dentro de ellos ocurren cuatro eventos principales, además del propio Sprint, que actúa como un contenedor de tiempo (*Time-Box*).

- **Sprint Planning (Planificación):**
 - **Propósito:** Definir el **Sprint Goal** (el objetivo unificado del ciclo) y seleccionar las tareas del *Product Backlog* que se realizarán para lograr ese objetivo, creando el *Sprint Backlog*.
 - **Duración:** *Time-Boxed* a un máximo de 8 horas para un Sprint de un mes (proporcionalmente menos para Sprints más cortos).
 - **Resultado:** Un *Sprint Goal* claro y el *Sprint Backlog* (el plan para el equipo de desarrollo).
- **Daily Scrum (Reunión Diaria/Daily Stand-up):**
 - **Propósito:** Inspeccionar el progreso hacia el *Sprint Goal* y adaptar el *Sprint Backlog* para el siguiente día de trabajo. Sirve para alinearse y detectar impedimentos.
 - **Duración:** Estrictamente *Time-Boxed* a **15 minutos**.
 - **Contenido:** El equipo de desarrollo responde (o simplemente dialoga sobre): **¿Qué hice ayer que ayudó al Equipo Scrum a lograr el *Sprint Goal*? ¿Qué haré hoy para ayudar al Equipo Scrum a lograr el *Sprint Goal*? ¿Hay algún impedimento que me impida a mí o al Equipo Scrum lograr el *Sprint Goal*?**
- **Sprint Review (Revisión):**
 - **Propósito:** Presentar el **Incremento** (el software funcional) del Sprint a los *stakeholders* y al *Product Owner* para recibir *feedback* y determinar las futuras adaptaciones al *Product Backlog*.
 - **Duración:** *Time-Boxed* a un máximo de 4 horas para un Sprint de un mes.
 - **Resultado:** *Feedback* valioso que informa las prioridades del *Product Backlog* para los siguientes Sprints.
- **Sprint Retrospective (Retrospectiva):**
 - **Propósito:** Inspeccionar al **Equipo Scrum y el proceso de trabajo** en sí mismo, identificando áreas de mejora. **Es el corazón de la mejora continua y el mecanismo de adaptación de Scrum.**
 - **Duración:** *Time-Boxed* a un máximo de 3 horas para un Sprint de un mes.
 - **Participantes:** Solo el Equipo Scrum (PO, SM y Developers) se reúne a **puertas cerradas** para hablar con honestidad sobre: **¿Qué salió bien? ¿Qué podemos mejorar como equipo? ¿Qué vamos a implementar como mejora en el siguiente Sprint?**
 - **Resultado:** Un plan de acción (una o dos mejoras específicas) para implementar en el siguiente Sprint.



9) Artefactos de Scrum (documentos de valor)

Los artefactos de Scrum son representaciones del trabajo o el valor que son utilizados para maximizar la transparencia y las oportunidades de inspección y adaptación.

1. **Product Backlog:**

- **Descripción:** Una lista **ordenada** y **viva** de todo lo que se sabe que es necesario para que el producto exista. Contiene características, funciones, requisitos, mejoras y correcciones de errores.
- **Naturaleza:** Es un documento dinámico. Cambia constantemente a medida que el mercado, la tecnología y el *feedback* del cliente evolucionan. Solo el *Product Owner* es responsable de su orden.
- **Concepto clave:** El *Product Backlog* nunca está "terminado", solo se refina.

2. **Sprint Backlog:**

- **Descripción:** El conjunto de ítems del *Product Backlog* seleccionados para el Sprint actual, más el plan detallado para entregar el **Incremento** y lograr el **Sprint Goal**.
- **Naturaleza:** Es un plan que pertenece y es gestionado únicamente por los *Developers*.
- **Concepto clave:** Es un plan flexible; se actualiza diariamente y puede ser renegociado con el *Product Owner* si se descubre que el *Sprint Goal* está en riesgo.

3. **Incremento:**

- **Descripción:** El resultado acumulado de todo el trabajo completado durante un Sprint y

todos los Sprints anteriores.

- **Naturaleza:** Debe ser **software que funcione** y cumpla con la **Definition of Done (DoD)**, lo que significa que tiene la calidad necesaria para ser entregado a los usuarios finales (aunque el PO decide si se entrega o no).
- **Concepto clave:** Cada incremento es aditivo y debe ser usable, asegurando que todos los incrementos trabajen juntos.

10) Agilidad en la actualidad (2026): equipos remotos e híbridos

El principio ágil de "**conversación cara a cara**" se escribió en un contexto donde el trabajo en la misma oficina era la norma. Hoy, con la proliferación de equipos distribuidos globalmente, este principio ha evolucionado para centrarse en la **calidad** de la comunicación síncrona y asíncrona, y el uso estratégico de herramientas digitales.

Herramientas Clave en la Agilidad Moderna:

- **Gestión del Flujo (Backlogs):** Herramientas como **Jira, Trello o Monday** se utilizan para mantener la transparencia y el estado del *Product Backlog* y del *Sprint Backlog*, asegurando que todos los miembros, sin importar su ubicación, vean el mismo progreso.
- **Comunicación Síncrona (Diaria):** Plataformas como **Slack o Microsoft Teams** han reemplazado gran parte de las conversaciones de pasillo y permiten las *Daily Scrum* virtuales, manteniendo la inmediatez.
- **Colaboración y Retrospectivas:** Las **pizarras virtuales** como **Miro o Mural** son esenciales para las *Sprint Retrospectives* y planificaciones, replicando la experiencia de un espacio de trabajo compartido y facilitando la colaboración visual a distancia.

El Desafío Central de la Agilidad 2026:

El mayor obstáculo en entornos remotos e híbridos es mantener la **cultura de equipo** y la **confianza** sin la interacción física constante. Para contrarrestar esto, la agilidad hoy requiere:

- **Responsabilidad Individual Extrema:** El equipo autogestionado debe demostrar un alto nivel de compromiso y cumplimiento sin una supervisión física directa.
- **Documentación Mínima pero Efectiva:** La necesidad de documentar decisiones y arquitecturas clave, aunque mínima, aumenta, ya que no todas las conversaciones son "escuchadas" por el equipo completo. La documentación se convierte en el "pegamento" de la colaboración asíncrona.

Actividades de la Semana 1

Actividad Práctica: "La Fábrica de Aviones"

1. Dividir a los alumnos en grupos de 5.
2. **Fase 1 (Cascada):** Tienen 10 minutos para diseñar y escribir el manual de construcción del "mejor avión de papel". No pueden tocar el papel hasta el final.
3. **Fase 2 (Agilidad):** Tienen 3 ciclos de 3 minutos. En cada ciclo deben hacer un prototipo, volarlo, ver qué falló y mejorar el diseño en el siguiente ciclo.
4. **Reflexión:** ¿Cuál avión voló mejor? ¿En qué modelo se sintieron más frustrados?